

Microservices: Yesterday, Today, and Tomorrow

Nicola Dragoni, Saverio Giallorenzo, Alberto Lluch Lafuente,
Manuel Mazzara, Fabrizio Montesi, Ruslan Mustafin, and Larisa Safina

Abstract Microservices is an architectural style inspired by service-oriented computing that has recently started gaining popularity. Before presenting the current state of the art in the field, this chapter reviews the history of software architecture, the reasons that led to the diffusion of objects and services first, and microservices later. Finally, open problems and future challenges are introduced. This survey primarily addresses newcomers to the discipline, while offering an academic viewpoint on the topic. In addition, we investigate some practical issues and point out a few potential solutions.

1 Introduction

The mainstream languages for the development of server-side applications, like Java, C/C++, and Python, provide abstractions to break down the complexity of programs into modules. However, these languages are designed for the creation of *single executable artefacts*, also called *monoliths*, and their modularisation abstractions rely on the sharing of resources of the same machine (memory, databases, files). Since the modules of a monolith depend on said shared resources, they are not independently executable.

N. Dragoni • A.L. Lafuente
Technical University of Denmark, Kgs. Lyngby, Denmark
e-mail: ndra@dtu.dk; albl@dtu.dk

S. Giallorenzo
Department of Computer Science and Engineering, INRIA/University of Bologna, Bologna, Italy
e-mail: saverio.giallorenzo@gmail.com

M. Mazzara (✉) • R. Mustafin • L. Safina
Innopolis University, Innopolis, Russia
e-mail: m.mazzara@innopolis.ru; r.mustafin@innopolis.ru; l.safina@innopolis.ru

F. Montesi
University of Southern Denmark, Odense M, Denmark
e-mail: fmontesi@imada.sdu.dk

Definition 1 (Monolith) A monolith is a software application whose modules cannot be executed independently.

This makes monoliths difficult to use in distributed systems without specific frameworks or ad hoc solutions such as Network Objects [7], RMI [40], or CORBA [67]. However, even these approaches still suffer from the general issues that affect monoliths; below we list the most relevant ones (we label issues **I** n):

- I|1 large-size monoliths are difficult to maintain and evolve due to their complexity. Tracking down bugs requires long perusals through their code base.
- I|2 Monoliths also suffer from the “dependency hell” [56], in which adding or updating libraries results in inconsistent systems that either do not compile/run or, worse, misbehave.
- I|3 Any change in one module of a monolith requires rebooting the whole application. For large-sized projects, restarting usually entails considerable downtimes, hindering the development, testing, and maintenance of the project.
- I|4 Deployment of monolithic applications is usually suboptimal due to conflicting requirements on the constituent models’ resources: some can be memory-intensive, others computational-intensive, and others require ad hoc components (e.g. SQL-based rather than graph-based databases). When choosing a deployment environment, the developer must compromise with a one-size-fits-all configuration, which is either expensive or suboptimal with respect to the individual modules.
- I|5 Monoliths limit scalability. The usual strategy for handling increments of inbound requests is to create new instances of the same application and to split the load amongst said instances. However, it could be the case that the increased traffic stresses only a subset of the modules, making the allocation of the new resources for the other components inconvenient.
- I|6 Monoliths also represent a technology lock-in for developers, which are bound to use the same language and frameworks of the original application.

The *microservice* architectural style [34] has been proposed to cope with such problems. In our definition of microservice, we use the term “cohesive” [2, 6, 10, 25, 44] to indicate that a service implements only functionalities strongly related to the concern that it is meant to model.

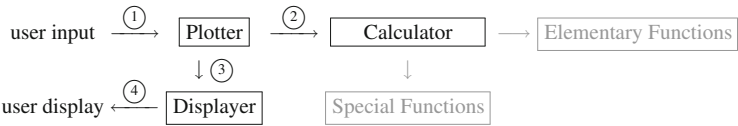
Definition 2 (Microservice) A microservice is a cohesive, independent process interacting via messages.

As an example, consider a service intended to compute calculations. To call it a microservice, it should provide arithmetic operations requestable via messages, but it should not provide other (possibly loosely related) functionalities like plotting and displaying of functions.

From a technical point of view, microservices should be independent components conceptually deployed in isolation and equipped with dedicated memory persistence tools (e.g. databases). Since all the components of a microservice architecture are microservices, its distinguishing behaviour derives from the composition and coordination of its components via messages.

Definition 3 (Microservice Architecture) A microservice architecture is a distributed application where all its modules are microservices.

To give an example of a microservice architecture, let us assume that we want to provide a functionality that plots the graph of a function. We also assume the presence of two microservices: Calculator and Displayer. The first is the calculator microservice mentioned above; the second renders and displays images. To fulfil our goal, we can introduce a new microservice, called Plotter, that orchestrates Calculator to calculate the shape of the graph and that invokes Displayer to render the calculated shape. Below, we report (in black) a depiction of the workflow of such a microservice architecture.



The developers of the architecture above can focus separately on implementing the basic microservice functionalities, i.e. the Calculator and the Displayer. Finally, they can implement the behaviour of the distributed application with the Plotter that ① takes the function given by a user, ② interacts with the Calculator to compute a symbolic representation of the graph of the function, and finally ③ requests the Displayer to show the result back to the user ④. To illustrate how the microservice approach scales by building on pre-existing microservice architectures, in the figure above we drew the Calculator orchestrating two extra microservices (in grey) that implement mathematical Elementary and Special Functions.

The microservice architectural style does not favour or forbid any particular programming paradigm. It provides a guideline to partition the components of a distributed application into independent entities, each addressing one of its concerns. This means that a microservice, provided it offers its functionalities via message passing, can be internally implemented with any of the mainstream languages cited in the beginning of this section.

The principle of microservice architectures assists project managers and developers: it provides a guideline for the design and implementation of distributed applications. Following this principle, developers focus on the implementation and testing of a few, cohesive functionalities. This holds also for higher-level microservices, which are concerned with coordinating the functionalities of other microservices.

We conclude this section with an overview, detailed in greater depth in the remainder of the chapter, on how microservices cope with the mentioned issues of monolithic applications (below, **S***ln* is a solution to issue **I***ln*).

- S|1 Microservices implement a limited amount of functionalities, which makes their code base small and inherently limits the scope of a bug. Moreover, since microservices are independent, a developer can directly test and investigate their functionalities in isolation with respect to the rest of the system.
- S|2 It is possible to plan *gradual transitions* to new versions of a microservice. The new version can be deployed “next” to the old one, and the services that depend on the latter can be gradually modified to interact with the former. This fosters *continuous integration* [33] and greatly eases software maintenance.
- S|3 As a consequence of the previous item, changing a module of a microservice architecture does not require a complete reboot of the whole system. The reboot regards only the microservices of that module. Since microservices are small in size, programmers can develop, test, and maintain services experiencing only very short redeployment downtimes.
- S|4 Microservices naturally lend themselves to containerisation [56], and developers enjoy a high degree of freedom in the configuration of the deployment environment that best suits their needs (both in terms of costs and quality of service).
- S|5 Scaling a microservice architecture does not imply a duplication of all its components, and developers can conveniently deploy/dispose instances of services with respect to their load [36].
- S|6 The only constraint imposed on a network of interoperating microservices is the technology used to make them communicate (media, protocols, data encodings). Apart from that, microservices impose no additional lock-in and developers can freely choose the optimal resources (languages, frameworks, etc.) for the implementation of each microservice.

In the remainder of this chapter, in Sect. 2, we give a brief account of the evolution of distributed architectures until their recent incarnation in the microservice paradigm. Then, we detail the problems that microservices can solve and their proposed solutions in the form of microservice architectures. In Sect. 3, we detail the current solutions for developing microservice architectures and how microservices affect the process of software design, development, testing, and maintenance. In Sect. 4, we discuss the open challenges and the desirable tools for programming microservice architecture. In Sect. 5, we draw overall conclusions.

2 Yesterday

Architecture is what allows systems to evolve and provide a certain level of service throughout their life cycle. In software engineering, architecture is concerned with providing a bridge between system functionality and requirements for quality attributes that the system has to meet. Over the past several decades, software architecture has been thoroughly studied, and as a result software engineers have come up with different ways to compose systems that provide broad functionality

and satisfy a wide range of requirements. In this section, we provide an overview of the work on software architectures from the early days to the advent of microservices.

2.1 *From the Early Days to Object-Oriented Design Patterns*

The problems associated with large-scale software development were first experienced around the 1960s [11]. The 1970s saw a huge rise of interest from the research community for software design and its implications on the development process. At the time, the design was often considered as an activity not associated with the implementation itself and therefore requiring a special set of notations and tools. Around the 1980s, the full integration of design into the development processes contributed towards a partial merge of these two activities, thus making it harder to make neat distinctions. References to the concept of software architecture also started to appear around the 1980s. However, a solid foundation on the topic was only established in 1992 by Perry and Wolf [70]. Their definition of software architecture was distinct from software design, and since then it has generated a large community of researchers studying the notion and the practical applications of software architecture, allowing the concepts to be widely adopted by both industry and academia.

This spike of interest contributed to an increase in the number of existing software architecture patterns (or generally called *styles*), so that some form of classification was then required. This problem was tackled in one of the most notable works in the field, the book *Software Architecture: Perspectives on an Emerging Discipline* by Garlan and Shaw [75]. Bosch's work [8] provides a good overview of the current research state in software engineering and architecture. Since its appearance in the 1980s, software architecture has developed into a mature discipline making use of notations, tools, and several techniques. From the pure, and occasionally speculative, realm of academic basic research, it has made the transition into an element that is essential to industrial software construction.

The advent and diffusion of object orientation, starting from the 1980s and in particular in the 1990s, brought its own contribution to the field of software architecture. The classic by Gamma et al. [37] covers the design of object-oriented software and how to translate it into code presenting a collection of recurring solutions, called *patterns*. This idea is neither new nor exclusive to software engineering, but the book is the first compendium to popularise the idea on a large scale. In the pre-Gamma era, patterns for object-oriented (OO) solutions were already used: a typical example of an architectural design pattern in object-oriented programming is the Model-View-Controller (MVC) [32], which has been one of the seminal insights in the early development of graphical user interfaces.

2.2 *Service-Oriented Computing*

Attention to *separation of concerns* has recently led to the emergence of the so-called component-based software engineering (CBSE) [77], which has given better control over design, implementation, and evolution of software systems. The last decade has seen a further shift towards the concept of service first [81] and the natural evolution to microservices afterwards.

Service-Oriented Computing (SOC) is an emerging paradigm for distributed computing and e-business processing that finds its origin in object-oriented and component computing. It has been introduced to harness the complexity of distributed systems and to integrate different software applications [51]. In SOC, a program—called a *service*—offers functionalities to other components, accessible via message passing. Services decouple their interfaces (i.e. how other services access their functionalities) from their implementation. On top of that, specific workflow languages are then defined in order to orchestrate the complex actions of services (e.g. WS-BPEL [66]). These languages share ideas with some well-known formalisms from concurrency theory, such as CCS and the π -calculus [57, 58]. This aspect fostered the development of formal models for better understanding and verifying service interactions, ranging from foundational process models of SOC [41, 50, 53] to theories for the correct composition of services [9, 45, 46]. In [86] a classification of approaches for business modelling puts this research in perspective.

The benefits of service-orientation are as follows:

- **Dynamism**—New instances of the same service can be launched to split the load on the system.
- **Modularity and reuse**—Complex services are composed of simpler ones. The same services can be used by different systems.
- **Distributed development**—By agreeing on the interfaces of the distributed system, distinct development teams can develop partitions of it in parallel.
- **Integration of heterogeneous and legacy systems**—Services merely have to implement standard protocols to communicate.

2.3 *Second Generation of Services*

The idea of componentisation used in service orientation can be partially traced back to the object-oriented programming (OOP) literature; however, there are peculiar differences that led to virtually separate research paths and communities. As a matter of fact, SOC at the origin was—and still is—built on top of OOP languages, largely due to their broad diffusion in the early 2000s. However, the evolution of objects into services, and the relative comparisons, has to be treated carefully since the first focus

on encapsulation and information is hidden in a *shared-memory* scenario, while the second is built on the idea of independent deployment and *message passing*. It is therefore a paradigm shift, where both the paradigms share the common idea of componentisation. The next step is adding the notion of *business capability* and therefore focusing analysis and design on it so that the overall system architecture is determined on this basis.

The first “generation” of service-oriented architectures (SOA) defined daunting and nebulous requirements for services (e.g. discoverability and service contracts), and this hindered the adoption of the SOA model. Microservices are the second iteration on the concept of SOA and SOC. The aim is to strip away unnecessary levels of complexity in order to focus on the programming of simple services that effectively implement a single functionality. Like OO, the microservice paradigm needs ad hoc tools to support developers and naturally leads to the emergence of specific design patterns [74]. First and foremost, languages that embrace the service-oriented paradigm are needed (instead, for the most part, microservice architectures still use OO languages like Java and Javascript or functional ones). The same holds for the other tools for development support like testing suites, (API) design tools, etc.

3 Today

The microservice architecture appeared lately as a new paradigm for programming applications by means of the composition of small services, each running its own processes and communicating via light-weight mechanisms. This approach has been built on the concepts of SOA [51], brought from crossing-boundaries workflows to the application level and into the applications architectures, i.e. its service-oriented architecture and programming from the large to the small.

The term “microservices” was first introduced in 2011 at an architectural workshop as a way to describe the participants’ common ideas in software architecture patterns [34]. Until then, this approach had also been known under different names. For example, Netflix used a very similar architecture under the name of *fine-grained SOA* [84].

Microservices now are a new trend in software architecture, which emphasises the design and development of highly maintainable and scalable software. Microservices manage growing complexity by functionally decomposing large systems into a set of independent services. By making services completely independent in development and deployment, microservices emphasise loose coupling and high cohesion by taking modularity to the next level. This approach delivers all sorts of benefits in terms of maintainability, scalability, and so on. It also comes with a bundle of problems that are inherited from distributed systems and from SOA, its predecessor. The microservice architecture still shows distinctive characteristics that blend into something unique and different from SOA itself:

- **Size**—The size is comparatively small wrt. a typical service, supporting the belief that the architectural design of a system is highly dependent on the structural design of the organisation producing it. Idiomatic use of the microservice architecture suggests that if a service is too large, it should be split into two or more services, thus preserving granularity and maintaining focus on providing only a single business capability. This brings benefits in terms of service maintainability and extendability.
- **Bounded context**—Related functionalities are combined into a single business capability, which is then implemented as a service.
- **Independency**—Each service in microservice architecture is operationally independent from other services, and the only form of communication between services is through their published interfaces.

The key system characteristics for microservices are as follows:

- **Flexibility**—A system is able to keep up with the ever-changing business environment and is able to support all modifications that is necessary for an organisation to stay competitive on the market.
- **Modularity**—A system is composed of isolated components where each component contributes to the overall system behaviour rather than having a single component that offers full functionality.
- **Evolution**—A system should stay maintainable while constantly evolving and adding new features.

The microservice architecture gained popularity relatively recently and can be considered to be in its infancy since there is still a lack of consensus on what microservices actually are. M. Fowler and J. Lewis provide a starting ground by defining principal characteristics of microservices [34]. S. Newman [64] builds upon M. Fowler's article and presents recipes and best practices regarding some aspects of the aforementioned architecture. L. Krause in his work [47] discusses patterns and applications of microservices. A number of papers have also been published that describe details of design and implementation of systems using microservice architecture. For example, the authors of [48] present development details of a new software system for Nevada Research Data Center (NRDC) using the microservice architecture. M. Rahman and J. Gao in [38] describe an application of behaviour-driven development (BDD) to the microservice architecture in order to decrease the maintenance burden on developers and encourage the usage of acceptance testing.

3.1 Teams

Back in 1968, Melvin Conway proposed that an organisation's structure or, more specifically, its communication structure constrains a system's design such that the resulting design is a copy of the organisation's communication patterns [22]. The microservices approach is to organise cross-functional teams around services,

which in turn are organised around business capabilities [34]. This approach is also known as “you build, you run it” principle, first introduced by Amazon CTO Werner Vogels [39]. According to this approach, teams are responsible for full support and development of a service throughout its life cycle.

3.2 Total Automation

Each microservice may represent a single business capability that is delivered and updated independently and on its own schedule. Discovering a bug and/or adding a minor improvement does not have any impact on other services and on their release schedule (of course, as long as backwards compatibility is preserved and a service interface remains unchanged). However, to truly harness the power of independent deployment, one must utilise very efficient integration and delivery mechanisms. This being said, microservices are the first architecture developed in the post-continuous delivery era, and essentially microservices are meant to be used with continuous delivery and continuous integration, making each stage of delivery pipeline automatic. By using automated continuous delivery pipelines and modern container tools, it is possible to deploy an updated version of a service to production in a matter of seconds [52], which proves to be very beneficial in rapidly changing business environments.

3.3 Choreography over Orchestration

As discussed earlier, microservices may cooperate in order to provide more complex and elaborate functionalities. There are two approaches to establish this cooperation—orchestration [54] and choreography [69]. Orchestration requires a conductor—a central service that will send requests to other services and oversee the process by receiving responses. Choreography, on the other hand, assumes no centralisation and uses events and publish/subscribe mechanisms in order to establish collaboration. These two concepts are not new to microservices, but rather are inherited from the SOA world where languages such as WS-BPEL [66] and WS-CDL [82] have long represented the major references for orchestration and choreography, respectively (with vivid discussions between the two communities of supporters).

Prior to the advent of microservices and at the beginning of the SOA’s hype in particular, orchestration was generally more popular and widely adopted, due to its simplicity of use and easier ways to manage complexity. However, it clearly leads to service coupling and uneven distribution of responsibilities, and therefore some services have a more centralising role than others. Microservices’ culture of decentralisation and high degrees of independence represents instead the natural application scenario for the use of choreography as a means of achieving collab-

oration. This approach has indeed recently seen a renewed interest in connection with the broader diffusion of microservices in what can be called the *second wave of services*.

3.4 Impact on Quality and Management

In order to better grasp microservices, we need to understand the impact that this architecture has on some software quality attributes.

3.4.1 Availability

Availability is a major concern in microservices as it directly affects the success of a system. Given services independence, the whole system availability can be estimated in terms of the availability of the individual services that compose the system. Even if a single service is not available to satisfy a request, the whole system may be compromised and experience direct consequences. If we take service implementation, the more fault-prone a component is, the more frequently the system will experience failures. One would argue that small-size components lead to a lower fault density. However, it has been found by Hatton [42] and by Compton and Withrow [21] that small-size software components often have a very high fault density. On the other hand, El Emam et al. in their work [28] found that as size increases, so does a component's fault proneness. Microservices are prevented from becoming too large as idiomatic use of microservice architecture suggests that, as a system grows larger, microservices should be prevented from becoming overly complex by refining them into two or more different services. Thus, it is possible to keep optimal size for services, which may theoretically increase availability. On the other hand, spawning an increasing number of services will make the system fault-prone on the integration level, which will result in decreased availability due to the large complexity associated with making dozens of services instantly available.

3.4.2 Reliability

Given the distributed nature of the microservice architecture, particular attention should be paid to the reliability of message-passing mechanisms between services and to the reliability of the services themselves. Building the system out of small and simple components is also one of the rules introduced in [72], which states that in order to achieve a higher reliability, one must find a way to manage the complexities of a large system: building things out of simple components with clean interfaces is one way to achieve this. The greatest threat to microservices reliability lies in the domain of integration, and therefore when talking about microservices reliability, one should also mention integration mechanisms. One example of this assumption

being false is using a network as an integration mechanism and assuming network reliability is one of the first fallacies of distributed computing [73]. Therefore, in this aspect, microservices reliability is inferior to the applications that use in-memory calls. It should be noted that this downside is not unique only to microservices and can be found in any distributed system. When talking about messaging reliability, it is also useful to remember that microservices put restrictions on integration mechanisms. More specifically, microservices use integration mechanisms in a very straightforward way—by removing all functionality that is not related to the message delivering and focusing solely on reliable message delivery.

3.4.3 Maintainability

By nature, the microservice architecture is loosely coupled, meaning that there is a small number of links between services and services themselves being independent. This greatly contributes to the maintainability of a system by minimising the costs of modifying services, fixing errors, or adding new functionality. Despite all efforts to make a system as maintainable as possible, it is always possible to spoil maintainability by writing obscure and counterintuitive code [4]. As such, another aspect of microservices that can lead to increased maintainability is the above-mentioned “you build it, you run it” principle, which leads to a better understanding of a given service, its business capabilities and roles [20, 29].

3.4.4 Performance

The prominent factor that negatively impacts performance in the microservice architecture is communication over a network. The network latency is much greater than that of memory. This means that in-memory calls are much faster to complete than sending messages over the network. Therefore, in terms of communication, the performance will degrade compared to applications that use in-memory call mechanisms. Restrictions that microservices put on size also indirectly contribute to this factor. In more general architectures without size-related restrictions, the ratio of in-memory calls to the total number of calls is higher than in the microservice architecture, which results in less communication over the network. Thus, the exact amount of performance degradation will also depend on the system’s interconnectedness. As such, systems with well-bounded contexts will experience less degradation due to looser coupling and fewer messages sent.

3.4.5 Security

In any distributed system, security becomes a major concern. In this sense, microservices suffer from the same security vulnerabilities as SOA [5]. As microservices use REST mechanism and XML with JSON as main data-interchange formats,

particular attention should be paid to providing security of the data being transferred. This means adding additional overhead to the system in terms of additional encryption functionality. Microservices promote service reuse, and as such it is natural to assume that some systems will include third-party services. Therefore, an additional challenge is to provide authentication mechanisms with third-party services and ensure that the sent data is stored securely. In summary, microservices' security is impacted in a rather negative manner because one has to consider and implement additional security mechanisms to provide additional security functionality mentioned above.

3.4.6 Testability

Since all components in a microservice architecture are independent, each component can be tested in isolation, which significantly improves component testability compared to monolithic architecture. It also allows to adjust the scope of testing based on the size of changes. This means that with microservices, it is possible to isolate parts of the system that changed and parts that were affected by the change and to test them independently from the rest of the system. Integration testing, on the other hand, can become very tricky, especially when the system that is being tested is very large, and there are too many connections between components. It is possible to test each service individually, but anomalies can emerge from collaboration of a number of services.

4 Tomorrow

Microservices are so recent that we can consider their exploration to have just begun. In this section, we discuss interesting future directions that we envision will play key roles in the advancement of the paradigm.

The greatest strength of microservices comes from pervasive distribution: even the internal components of software are autonomous services, leading to loosely coupled systems and the other benefits previously discussed. However, from this same aspect (distribution) also comes its greatest weakness: programming distributed systems is inherently harder than monoliths. We now have to think about new issues. Some examples are how can we manage changes to a service that may have side effects on other services that it communicates with? How can we prevent attacks that exploit network communications?

4.1 *Dependability*

There are many pitfalls that we need to keep in mind when programming with microservices. In particular, preventing programming errors is hard. Consequently, building dependable systems is challenging.

4.1.1 Interfaces

Since microservices are autonomous, we are free to use the most appropriate technology for the development of each microservice. A disadvantage introduced by this practice is that different technologies typically have different means of specifying contracts for the composition of services (e.g. interfaces in Java, or WSDL documents in Web Services [19]). Some technologies do not even come with a specification language and/or a compatibility checker of microservices (Node.js, based on JavaScript, is a prime example).

Thus, where do we stand? Unfortunately, the current answer is informal documentation. Most services come with informal documents expressed in natural language that describe how clients should use the service. This makes the activity of writing a client very error-prone, due to potential ambiguities. Moreover, we have no development support tools to check whether service implementations actually implement their interfaces correctly.

As an attempt to fix this problem, there are tools for the formal specification of message types for data exchange, which one can use to define service interfaces independently of specific technologies. Then, these technology-agnostic specifications can be either compiled to language-specific interfaces—e.g. compiling an interface to a Java type—or used to check for well-typedness of messages (wrt. interfaces and independently of the transport protocol). Examples of tools offering these methodologies are Jolie [3, 55, 63], Apache Thrift [71], and Google’s Protocol Buffers [80]. However, it is still unclear how to adapt tools to implement the mechanical checking (at compile or execution time) of messages for some widespread architectural styles for microservices, such as REST [30], where interfaces are constrained to a fixed set of operations and actions are expressed on dynamic resource paths. A first attempt at bridging the world of technology-agnostic interfaces based on operations and REST is presented in [61], but checking for the correctness of the binding information between the two is still left as a manual task to the programmer. Another, and similar, problem is trying to apply static type-checking to dynamic languages (e.g. JavaScript and Jolie), which are largely employed in the development of microservices [1, 31, 59].

4.1.2 Behavioural Specifications and Choreographies

Having formally defined interfaces in the form of an API is not enough to guarantee the compatibility of services. This is because, during execution, services may engage in sessions during which they perform message exchanges in a precise order. If two services engage in a session and start performing incompatible I/O, this can lead to various problems. Examples include a client sending a message on a stream that was previously closed; deadlocks, when two services expect a message from one another without sending anything; or, a client trying to access an operation that is offered by a server only after a successful distributed authentication protocol with a third-party is performed.

Behavioural types are types that can describe the behaviour of services and can be used to check that two (or more) services have compatible actions. Session types are a prime example of behavioural types [45, 46]. Session types have been successfully applied to many contexts already, ranging from parallel to distributed computing. However, no behavioural type theory is widely adopted in practice yet. This is mainly because behavioural types restrict the kind of behaviours that programmers can write for services, limiting their applicability. An important example of a feature with space for improvement is nondeterminism. In many interesting protocols, like those for distributed agreement, execution is nondeterministic and depending on what happens at runtime, the participants have to react differently [68].

Behavioural interfaces are a hot topic right now and will likely play an important role in the future of microservices. We envision that they will also be useful for the development of automatic testing frameworks that check the communication behaviour of services.

4.1.3 Choreographies

Choreographies are high-level descriptions of the communications that we want to happen in a system in contrast with the typical methodology of defining the behaviour of each service separately. Choreographies are used in some models for behavioural interfaces, but they actually originate from efforts at the W3C of defining a language that describes the global behaviour of service systems [85]. Over the past decade, choreographies have been investigated for supporting a new programming paradigm called Choreographic Programming [60]. In Choreographic Programming, the programmer uses choreographies to program service systems and then a compiler is used to automatically generate compliant implementations. This yields a correctness-by-construction methodology, guaranteeing important properties such as deadlock freedom and lack of communication errors [14, 15, 62].

Choreographies may have an important role in the future of microservices, since they shrink the gap between requirements and implementations, making the programmer able to formalise the communications envisioned in the design phase of a software. Since the correctness of the compiler from choreographies to distributed

implementations is vital in this methodology, formal models are being heavily adopted to develop correct compilation algorithms [35]. However, a formalisation of how transparent mobility of processes from one protocol to the other is still missing. Moreover, it is still unclear how choreographies can be combined with flexible deployment models where nodes may be replicated or fail at runtime. An initial investigation on the latter is given in [49]. Also, choreographies are still somewhat limited in expressing nondeterministic behaviour, just like behavioural types.

4.1.4 Moving Fast with Solid Foundations

Behavioural types, choreographies, refinement types [78], and other models address the problem of specifying, verifying, and synthesising communication behaviours. However, there is still much to be discovered and developed on these topics. It is then natural to ask: do we really need to start these investigations from scratch? Or, can we hope to reuse results and structures from other well-established models in computer science?

A recent line of work suggests that a positive answer can be found by connecting behavioural types and choreographies to well-known logical models. A prominent example is a Curry-Howard correspondence between session types and the process model of π -calculus, given in [13] (linear logic propositions correspond to session types, and communications to proof normalisation in linear logic). This result has propelled many other results, amongst which are as follows: a logical reconstruction of behavioural types in classical linear logic that supports parametric polymorphism [83], type theories for integrating higher-order process models with functional computation [79], initial ideas for algorithms for extracting choreographies from separate service programs [16], a logical characterisation of choreography-based behavioural types [17], and explanations of how interactions amongst multiple services (multiparty sessions) are related to well-known techniques for logical reasoning [12, 18].

Another principle that we can use for the evolution of choreographic models is the established notion of computation. The minimal set of language features to achieve Turing completeness in choreographies is known [23]. More relevant in practice, this model was used to develop a methodology of procedural programming for choreographies, allowing for the writing of correct-by-construction implementations of divide-and-conquer distributed algorithms [24].

We can then conclude that formal methods based on well-known techniques seem to be a promising starting point for tackling the issue of writing correct microservice systems. This starting point gives us solid footing for exploring the more focused disciplines that we will need in the future, addressing problems like the description of coordination patterns amongst services. We envision that these patterns will benefit from the rich set of features that formal languages and process models have to offer, such as expressive type theories and logic. It is still unclear, however, how exactly these disciplines can be extended to naturally

capture the practical scenarios that we encounter in microservices. We believe that empirically investigating microservice programming will be beneficial in finding precise research directions in this regard.

4.2 *Trust and Security*

The microservice paradigm poses a number of trust and security challenges. These issues are certainly not new, as they apply to SOA and in general to distributed computing, but they become even more challenging in the context of microservices. In this section, we aim to discuss some of these key security issues.

4.2.1 Greater Surface Attack Area

In monolithic architectures, application processes communicate via internal data structures or internal communication (for instance, socket or RMI). The attack surface is usually also constrained to a single OS. On the contrary, the microservice paradigm is characterised by applications that are broken down into services that interact with each other through APIs exposed to the network. APIs are independent of machine architectures and even programming languages. As a result, they are exposed to more potential attacks than traditional subroutines or functionalities of a large application, which only interacted with other parts of the same application. Moreover, application internals (the microservices) have now become accessible from the external world. Rephrasing, this means that microservices can in principle send the attack surface of a given application through the roof.

4.2.2 Network Complexity

The microservices vision, based on the creation of many small independent applications interacting with each other, can result in complex network activity. This network complexity can significantly increase the difficulty in enforcing the security of the overall microservices-based application. Indeed, when a real-world application is decomposed, it can easily create hundreds of microservices, as seen in the architecture overview of Hailo, an online cab reservation application.¹ Such an intrinsic complexity determines an ever-increasing difficulty in debugging, monitoring, auditing, and forensic analysis of the entire application. Attackers could exploit this complexity to launch attacks against applications.

¹hailoapp.com.

4.2.3 Trust

Microservices, at least in this early stage of development, are often designed to completely trust each other. Considering a microservice trustworthy represents an extremely strong assumption in the “connectivity era”, where microservices can interact with each other in a heterogeneous and open way. An individual microservice may be attacked and controlled by a malicious adversary, compromising not only the single microservice but, more drastically, bringing down the entire application. As an illustrative real-world example, a subdomain of Netflix was recently compromised, and from that domain, an adversary can serve any content in the context of netflix.com. In addition, since Netflix allowed all users’ cookies to be accessed from any subdomain, a malicious individual controlling a subdomain was able to tamper with authenticated Netflix subscribers and their data [76]. Future microservices platforms need mechanisms to monitor and enforce the connections amongst microservices to confine the trust placed on individual microservices, limiting the potential damage if any microservice gets compromised.

4.2.4 Heterogeneity

The microservice paradigm brings heterogeneity (of distributed systems) to its maximum expression. Indeed, a microservices-based system can be characterised by a large number of autonomous entities that are not necessarily known in advance (again, trust issue); a large number of different administrative security domains, creating competition amongst providers of different services; a large number of interactions across different domains (through APIs); no common security infrastructure (different “Trusted Computing Base”); and, last but not least, no global system to enforce rules.

The research community is still far from adequately addressing the aforementioned security issues. Some recent works, like [76], show that some preliminary contribution is taking place. However, the challenge of building secure and trustworthy microservices-based systems is still more than open.

5 Conclusions

The microservice architecture is a style that has been increasingly gaining popularity in the last few years, both in academia and in the industrial world. In particular, the shift towards microservices is a sensitive matter for a number of companies involved in a major refactoring of their back-end systems [27].

Despite the fact that some authors present it from a *revolutionary* perspective, we have preferred to provide an *evolutionary* presentation to help the reader understand the main motivations that lead to the distinguishing characteristics of microservices and relate to well-established paradigms such as OO and SOA. With microservice

architecture being very recent, we have not found a sufficiently comprehensive collection of literature in the field, so that we felt the need to provide a starting point for newcomers to the discipline and offer the authors' viewpoint on the topic.

In this chapter, we have presented a (necessarily incomplete) overview of software architecture, mostly providing the reader with references to the literature and guiding him/her in our itinerary towards the advent of services and microservices. A specific arc has been given to the narrative, which necessarily emphasises some connections and some literature, and it is possibly too severe with other sources. For example, research contributions in the domain of the actor model [43] and software agents [65] have not been emphasised enough, and still modern distributed systems have been influenced by these communities too. This calls for a broader survey investigating relationships along this line. For information on the relation between microservices and scalability, the reader may refer to [26].

Acknowledgements Montesi was supported by Choreographies for Reliable and efficient Communication software (CRC), grant no. DFF-4005-00304 from the Danish Council for Independent Research. Giallorenzo was supported by the EU EIT Digital project SMALL. This work has been partially funded by an Erasmus Mundus Scholarship. We would like to thank Daniel Martin Johnston who played a major role in proofreading the final draft of the chapter and improving the quality of writing.

References

1. Akentev, E., Tchitchigin, A., Safina, L., Mazzara, M.: Verified type-checker for Jolie. <https://arxiv.org/pdf/1703.05186.pdf>
2. Allen, E.B., Khoshgoftaar, T.M., Chen, Y.: Measuring coupling and cohesion of software modules: an information-theory approach. In: Proceedings Seventh International Software Metrics Symposium, pp. 124–134 (2001)
3. Bandura, A., Kurilenko, N., Mazzara, M., Rivera, V., Safina, L., Tchitchigin, A.: Jolie community on the rise. In: 9th IEEE International Conference on Service-Oriented Computing and Applications, SOCA (2016)
4. Bass, L.: Software Architecture in Practice. Pearson Education India, New Delhi (2007)
5. Bass, L., Merson, P., O'Brien, L.: Quality attributes and service-oriented architectures. Department of Defense, Technical Report September (2005)
6. Bieman, J.M., Kang, B.-K.: Cohesion and reuse in an object-oriented system. In: Proceedings of the 1995 Symposium on Software Reusability, SSR '95, pp. 259–262. ACM, New York (1995)
7. Birrell, A., Nelson, G., Owicki, S., Wobber, E.: Network objects. SIGOPS Oper. Syst. Rev. **27**(5), 217–230 (1993)
8. Bosch, J.: Software architecture: the next step. In: Software Architecture, pp. 194–199. Springer, Berlin (2004)
9. Bravetti, M., Zavattaro, G.: Towards a unifying theory for choreography conformance and contract compliance. In: Software Composition, pp. 34–50. Springer, Berlin, Heidelberg (2007)
10. Briand, L.C., Daly, J.W., Wüst, J.K.: A unified framework for coupling measurement in object-oriented systems. IEEE Trans. Softw. Eng. **25**(1), 91–121 (1999)
11. Brooks, F.P.: The Mythical Man-Month, vol. 1995. Addison-Wesley, Reading (1975)

12. Caires, L., Pérez, J.A.: Multiparty session types within a canonical binary theory, and beyond. In: Formal Techniques for Distributed Objects, Components, and Systems - 36th IFIP WG 6.1 International Conference, FORTE 2016, Held as Part of the 11th International Federated Conference on Distributed Computing Techniques, DisCoTec 2016, Heraklion, Crete, June 6–9, 2016, Proceedings, pp. 74–95 (2016)
13. Caires, L., Pfenning, F.: Session types as intuitionistic linear propositions. In: CONCUR, pp. 222–236 (2010)
14. Carbone, M., Montesi, F.: Deadlock-freedom-by-design: multiparty asynchronous global programming. In: POPL, pp. 263–274 (2013)
15. Carbone, M., Honda, K., Yoshida, N.: Structured communication-centered programming for web services. *ACM Trans. Program. Lang. Syst.* **34**(2), 8 (2012)
16. Carbone, M., Montesi, F., Schürmann, C.: Choreographies, logically. In: CONCUR, pp. 47–62 (2014)
17. Carbone, M., Montesi, F., Schürmann, C., Yoshida, N.: Multiparty session types as coherence proofs. In: CONCUR, pp. 412–426 (2015)
18. Carbone, M., Lindley, S., Montesi, F., Schürmann, C., Wadler, P.: Coherence generalises duality: a logical explanation of multiparty session types. In: Desharnais, J., Jagadeesan, R. (eds.) 27th International Conference on Concurrency Theory (CONCUR 2016). *Leibniz International Proceedings in Informatics (LIPIcs)*, vol. 59, pp. 33:1–33:15. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Dagstuhl (2016). doi:[10.4230/LIPIcs.CONCUR.2016.33](https://doi.org/10.4230/LIPIcs.CONCUR.2016.33), ISBN:978-3-95977-017-0. <http://drops.dagstuhl.de/opus/volltexte/2016/6181>
19. Christensen, E., Curbera, F., Meredith, G., Weerawarana, S., et al.: Web services description language (wsdl) 1.1 (2001)
20. Cohen, J., Brown, E., DuRette, B., Teleki, S.: *Best Kept Secrets of Peer Code Review*. Smart Bear, Somerville (2006)
21. Compton, B.T., Withrow, C.: Prediction and control of ADA software defects. *J. Syst. Softw.* **12**(3), 199–207 (1990)
22. Conway, M.E.: How do committees invent. *Datamation* **14**(4), 28–31 (1968)
23. Cruz-Filipe, L., Montesi, F.: Choreographies, computationally. In: CoRR, abs/1510.03271 (2015)
24. Cruz-Filipe, L., Montesi, F.: Choreographies, divided and conquered. In: CoRR, abs/1602.03729 (2016)
25. Dhami, H.: Quantitative models of cohesion and coupling in software. *J. Syst. Softw.* **29**(1), 65–74 (1995)
26. Dragoni, N., Lanese, I., Larsen, S.T., Mazzara, M., Mustafin, R., Safina, L.: Microservices: how to make your application scale. In: A.P. Ershov Informatics Conference (the PSI Conference Series, 11th edn.). Springer, Berlin (2017)
27. Dragoni, N., Dustdar, S., Larse, S.T., Mazzara, M.: Microservices: migration of a mission critical system. <https://arxiv.org/abs/1704.04173>
28. El Emam, K., Goel, N., Melo, W., Lounis, H., Rai, S.N., et al.: The optimal class size for object-oriented software. *IEEE Trans. Softw. Eng.* **28**(5), 494–509 (2002)
29. Fagan, M.: Design and code inspections to reduce errors in program development. In: *Software Pioneers*, pp. 575–607. Springer, Berlin (2002)
30. Fielding, R.T.: *Architectural styles and the design of network-based software architectures*. PhD thesis, University of California, Irvine (2000)
31. Flow: A static type checker for JavaScript. <https://flowtype.org/>
32. Fowler, M.: *Patterns of Enterprise Application Architecture*. Addison-Wesley Longman, Boston (2002)
33. Fowler, M., Foemmel, M.: Continuous integration. <https://www.thoughtworks.com/continuous-integration> (2006)
34. Fowler, M., Lewis, J.: Microservices. <http://martinfowler.com/articles/microservices.html> (2014)
35. Gabrielli, M., Giallorenzo, S., Montesi, F.: Applied choreographies. In: CoRR, abs/1510.03637 (2015)

36. Gabbriellini, M., Giallorenzo, S., Guidi, C., Mauro, J., Montesi, F.: Self-reconfiguring microservices. In: *Theory and Practice of Formal Methods*, pp. 194–210. Springer, Berlin (2016)
37. Gamma, E., Helm, R., Johnson, R., Vlissides, J.: *Design Patterns: Elements of Reusable Object-Oriented Software*. Pearson Education India, New Delhi (1995)
38. Gao, J., Rahman, M.: A reusable automated acceptance testing architecture for microservices in behavior-driven development. In: *Proceedings of the 2015 IEEE Symposium on Service-Oriented System Engineering (SOSE '15)*, pp. 321–325. IEEE Computer Society, Washington, DC (2015). doi:[10.1109/SOSE.2015.55](https://doi.org/10.1109/SOSE.2015.55), ISBN:978-1-4799-8356-8. <http://dx.doi.org/10.1109/SOSE.2015.55>
39. Gray, J.: A conversation with Werner Vogels. *ACM Queue* **4**(4), 14–22 (2006)
40. Grosso, W.: *Java RMI*, 1st edn. O'Reilly & Associates, Inc., Newton (2001)
41. Guidi, C.: *Formalizing languages for service oriented computing*. Ph.D. thesis, University of Bologna (2007)
42. Hatton, L.: Reexamining the fault density-component size connection. *IEEE Softw.* **14**(2), 89–97 (1997)
43. Hewitt, C., Bishop, P., Steiger, R.: A universal modular actor formalism for artificial intelligence. In: *Proceedings of the 3rd International Joint Conference on Artificial Intelligence, IJCAI'73*, pp. 235–245. Morgan Kaufmann, Burlington (1973)
44. Hitz, M., Montazeri, B.: *Measuring coupling and cohesion in object-oriented systems*. Citeseer (1995)
45. Honda, K., Vasconcelos, V., Kubo, M.: Language primitives and type disciplines for structured communication-based programming. In: *Proceedings of the 7th European Symposium on Programming: Programming Languages and Systems*, pp. 22–138 (1998)
46. Honda, K., Yoshida, N., Carbone, M.: Multiparty asynchronous session types. *J. ACM* **63**(1), 9 (2016). Also: *POPL*, pp. 273–284 (2008)
47. Krause, L.: *Microservices: Patterns and Applications*, 1 edn. Lucas Krause, Paris (2014). 1 April 2015
48. Le, V.D., Neff, M.M., Stewart, R.V., Kelley, R., Fritzing, E., Dascalu, S.M., Harris, F.C.: Microservice-based architecture for the NRDC. In: *2015 IEEE 13th International Conference on Industrial Informatics (INDIN)*, July 2015, pp. 1659–1664
49. López, H.A., Nielson, F., Nielson, H.R.: Enforcing availability in failure-aware communicating systems. In: *Formal Techniques for Distributed Objects, Components, and Systems - 36th IFIP WG 6.1 International Conference, FORTE 2016, Held as Part of the 11th International Federated Conference on Distributed Computing Techniques, DisCoTec 2016, Heraklion, Crete, June 6–9, 2016, Proceedings*, pp. 195–211 (2016)
50. Lucchi, R., Mazzara, M.: A pi-calculus based semantics for WS-BPEL. *J. Logic Algebraic Program.* **70**(1), 96–118 (2007)
51. MacKenzie, M.C., Laskey, K., McCabe, F., Brown, P.F., Metz, R., Hamilton, B.A.: Reference model for service oriented architecture 1.0. OASIS Standard, 12 Oct 2006
52. Mauro, T.: Adopting microservices at netflix: lessons for team and process design. <http://nginx.com/blog/adopting-microservices-at-netflix-lessons-for-team-and-process-design/> (2015)
53. Mazzara, M.: *Towards abstractions for web services composition*. Ph.D. thesis, University of Bologna (2006)
54. Mazzara, M., Govoni, S.: *A Case Study of Web Services Orchestration*, pp. 1–16. Springer, Berlin, Heidelberg (2005)
55. Mazzara, M., Montesi, F., Guidi, C., Lanese, I.: *Microservices: a language-based approach*. In: *Present and Ulterior Software Engineering*. Springer, Berlin (2017)
56. Merkel, D.: Docker: lightweight Linux containers for consistent development and deployment. *Linux J.* **2014**(239), 2 (2014)
57. Milner, R.: *A Calculus of Communicating Systems*. Lecture Notes in Computer Science, vol. 92. Springer, Berlin (1980)
58. Milner, R., Parrow, J., Walker, D.: A calculus of mobile processes, I and II. *Inf. Comput.* **100**(1), 1–40, 41–77 (1992)

59. Mingela, B., Troshkov, N., Mazzara, M., Safina, L., Tchitchigin, A.: Towards static type-checking for Jolie. <https://arxiv.org/pdf/1702.07146.pdf>
60. Montesi, F.: Choreographic programming. Ph.D. thesis, IT University of Copenhagen. http://www.fabriziomontesi.com/files/choreographic_programming.pdf (2013)
61. Montesi, F.: Process-aware web programming with Jolie. *Sci. Comput. Program.* **130**, 69–96 (2016)
62. Montesi, F., Yoshida, N.: Compositional choreographies. In: *CONCUR*, pp. 425–439 (2013)
63. Montesi, F., Guidi, C., Zavattaro, G.: Service-Oriented Programming with Jolie. In: *Web Services Foundations*, pp. 81–107. Springer, Berlin (2014)
64. Newman, S.: *Building Microservices*. O'Reilly Media, Sebastopol (2015)
65. Nwana, H.S.: Software agents: an overview. *Knowl. Eng. Rev.* **11**, 205–244, 9 (1996)
66. OASIS: Web Services Business Process Execution Language. <http://docs.oasis-open.org/wsbpel/2.0/wsbpel-v2.0.html>
67. OMG: Common Object Request Broker Architecture. <http://www.omg.org/spec/CORBA/>
68. Ongaro, D., Ousterhout, J.K.: In search of an understandable consensus algorithm. In: *2014 USENIX Annual Technical Conference, USENIX ATC '14*, Philadelphia, PA, 19–20 June 2014, pp. 305–319
69. Peltz, C.: Web services orchestration and choreography. *Computer* **36**(10), 46–52 (2003)
70. Perry, D.E., Wolf, A.L.: Foundations for the study of software architecture. *ACM SIGSOFT Softw. Eng. Notes* **17**(4), 40–52 (1992)
71. Prunicki, A.: *Apache thrift* (2009)
72. Raymond, E.S.: *The Art of Unix Programming*. Addison-Wesley Professional, Indianapolis (2003)
73. Rotem-Gal-Oz, A.: Fallacies of distributed computing explained, p. 20. <http://www.rgoarchitects.com/Files/fallacies.pdf> (2006)
74. Safina, L., Mazzara, M., Montesi, F., Rivera, V.: Data-driven workflows for microservices (genericity in Jolie). In: *Proc. of The 30th IEEE International Conference on Advanced Information Networking and Applications (AINA)* (2016)
75. Shaw, M., Garlan, D.: *Software architecture: perspectives on an emerging discipline*, vol. 1. Prentice Hall, Englewood Cliffs (1996)
76. Sun, Y., Nanda, S., Jaeger, T.: Security-as-a-service for microservices-based cloud applications. In: *Proceedings of the 2015 IEEE 7th International Conference on Cloud Computing Technology and Science (CloudCom), CLOUDCOM '15*, pp. 50–57. IEEE Computer Society, Washington, DC (2015)
77. Szyperski, C.: *Component Software: Beyond Object-Oriented Programming*, 2nd edn. Addison-Wesley Longman Publishing Co., Inc., Boston (2002)
78. Tchitchigin, A., Safina, L., Mazzara, M., Elwakil, M., Montesi, F., Rivera, V.: Refinement types in Jolie. In: *Spring/Summer Young Researchers Colloquium on Software Engineering, SYRCoSE* (2016)
79. Toninho, B., Caires, L., Pfenning, F.: Higher-order processes, functions, and sessions: a monadic integration. In: *Programming Languages and Systems - 22nd European Symposium on Programming, ESOP 2013, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2013, Rome, March 16–24, 2013. Proceedings*, pp. 350–369 (2013)
80. Varda, K.: Protocol buffers: Google's data interchange format. Google Open Source Blog, Available at least as early as July 2008
81. W3C: Web services architecture. <http://www.w3.org/TR/ws-arch/>
82. W3C: Web services choreography description language. <https://www.w3.org/TR/ws-cdl-10/>
83. Wadler, P.: Propositions as sessions. *J. Funct. Program.* **24**(2–3), 384–418 (2014). Also: *ICFP*, pp. 273–286 (2012)
84. Wang, A., Tonse, S.: Announcing ribbon: tying the netflix mid-tier services together, January 2013. <http://techblog.netflix.com/2013/01/announcing-ribbon-tying-netflix-mid.html>
85. Web Services Choreography Working Group et al.: *Web services choreography description language* (2002)

86. Yan, Z., Mazzara, M., Cimpian, E., Urbanec, A.: Business process modeling: classifications and perspectives. In: Business Process and Services Computing: 1st International Working Conference on Business Process and Services Computing, BPSC 2007, September 25–26, 2007, Leipzig, p. 222 (2007)